AARHUS UNIVERSITY
DEPARTMENT OF ENGINEERING

COMPUTER ENGINEERING

Term:	SPRING 2024
Examination:	Software Architecture
Date & Time:	17 June 2023, 9:00 – 11:00
Duration:	2 hours

- This is the exam for the lecture Software Architecture. It consists of 15 questions. You have 2 hours to answer all of them. Strategically, you should first scan the exam paper quickly in its entirety, and identify the groups of questions that seem easiest to you. ***You should answer those first*** and then move on to the others.
- For each question it is indicated how and where the answer must be provided. There are questions that require writing the result of a calculation, and questions that require you to write C or C++ code and drawings, in particular. Only what is written or drawn inside the specified areas is considered as your answer.
- **If you are in doubt what is expected, please ask!**
- Each question has a weight **[w]**, where **w** is an integer number, specifying the number of points awarded for that question. You can find the weight at the top right corner of each question. The total number of points of all questions together is 100.

Question 1 **[7]**

You are given the following concrete classes `Apple` and `Pear`, with function members

- `for_eating` that yields whether this fruit is intended for eating,
- `set_origin` to set the origin where the fruit was grown, and
- `as_string` to provide a textual representation of the fruit.

Provide an abstract class named `Fruit` that abstracts the functionality of both classes. Hint: Make sure that it is clear which members are virtual and which aren't.

```
#ifndef APPLE_H
#define APPLE_H

#include <string>
using namespace std;

class Apple {
private:
  int origin;
  int size;
  bool sour;
public:
  Apple(int, int, bool);
  ~Apple();
  bool for_eating();
  void set_origin(int);
  string as_string();
};

#endif
```

```
#ifndef PEAR_H
#define PEAR_H

#include <string>
using namespace std;

class Pear {
private:
  int origin;
  int weight;
  bool sweet;
public:
  Pear(int, int, bool);
  ~Pear();
  bool for_eating();
  void set_origin(int);
  string as_string();
};

#endif
```

Answer:

```
#ifndef FRUIT_H
#define FRUIT_H

#include <string>
using namespace std;

class Fruit {

public:

    virtual ~Fruit() {}

    virtual bool for_eating() = 0;

    virtual void set_origin(int) = 0;

    virtual string as_string() = 0;

};

#endif
```

Question 2 **[8]**

Let the following part of an implementation of a library system be given.

```cpp
#include "Book.h"

class Library {
private:
  int site;
  Book** inventory;
public:
  Library(int, int);
  ~Library();
  void add_book(Book*);
  bool loan(Stamp*);
  int get_site();
  string get_book_name(Book*);
};
```

```cpp
#include "Stamp.h"

class Book {
private:
  Stamp* stamp;
  bool present;
public:
  Book(Stamp*);
  ~Book();
  Stamp* get_stamp();
  void new_stamp(Stamp*);
  bool is_present();
};
```

The class `Library` depends directly on the lower level class `Book`,

Library → Book

Suppose, you do not want `Library` to depend on `Book`. Complete the classes `Book`, `IItem`, and `Library` below, applying dependency inversion, so that

Library → IItem ← Book

Answer:

```cpp
#include "IItem.h"
#include "Stamp.h"
#include <string>

class Library {
private:
  int site;
  IItem** inventory;
public:
  Library(int, int);
  ~Library();
  void add_book(IItem*);
  bool loan(Stamp*);
  int get_site();
  string get_book_name(IItem*);
};
```

```cpp
#include "IItem.h"

class Book : public IItem {
private:
  Stamp* stamp;
  bool present;
public:
  Book(Stamp*);
  ~Book();

  Stamp* get_stamp() override;

  void new_stamp(Stamp*) override;

  bool is_present() override;
};
```

```cpp
#include "Stamp.h"

class IItem {
public:

  virtual ~IItem() {}

  virtual Stamp* get_stamp() = 0;

  virtual void new_stamp(Stamp*) = 0;

  virtual bool is_present() = 0;
};
```

Question 3 **[9]**

You are given the following C++-implementation of an associative `int` array. Besides the constructor `Int_assoc(int s)` for creating an associative array of size `s` and a destructor `~Int_assoc()`, there is

- a function member `Int_assoc* set(string*, int)` for setting a value,
- a function member `int unset(string*, int)` for unsetting a value,
- and a function member `void apply(int (*func)(int, int), int)` transforming values of the associative array using the function `func` and the `int` value `v` passed as arguments.

```cpp
class Int_assoc {
private:
  int size;
  string** keys;
  int* values;
public:
  Int_assoc(int s): size(s), keys(new string*[s]) {
    this->values = new int[s];
  }
  ~Int_assoc() {
    delete[] keys;
    delete[] values;
  }
  Int_assoc* set(string* key, int value) {
    int p = -1;
    for (int k = 0; k < this->size; k++) {
      if (this->keys[k] == key) {
        this->values[k] = value;
        return this;
      }
      if (p < 0 && this->keys[k] == 0) {
        p = k;
      }
    }
    if (p >= 0) {
      this->keys[p] = key;
      this->values[p] = value;
    }
    return this;
  }
  int unset(string* key, int value) {
    int res = value;
    for (int k = 0; k < this->size; k++) {
      if (this->keys[k] == key) {
        this->keys[k] = 0;
        return values[k];
      }
    }
    return res;
  }
  void apply(int (*func)(int, int), int v) {
    for (int k = 0; k < this->size; k++) {
        int value = (*func)(this->values[k], v);
        this->values[k] = value;
    }
  }
};
```

The statement of Question 3 is continued on the next page.

Using a template create a new class `Assoc` which behaves like `Int_assoc` but for an arbitrary choice of type `T` instead of exclusively `int`.

Complete the class `Assoc` so that a variable of type `Assoc<int>` will behave just as a variable of type `Int_assoc`.

```
template <typename T>
class Assoc {
private:
  int size;

  string* * keys;

  T * values;
public:
  Assoc(int s): size(s), keys(new string*[s]) {
    this->values = new T[s];
  }
  ~Assoc() {
    delete[] keys;
    delete[] values;
  }
  Assoc<T> * set(string* key, T value) {

    int p = -1;
    for (int k = 0; k < this->size; k++) {
      if (this->keys[k] == key) {
        this->values[k] = value;
        return this;
      }
      if (p < 0 && this->keys[k] == 0) {
        p = k;
      }
    }
    if (p >= 0) {
      this->keys[p] = key;
      this->values[p] = value;
    }
    return this;
  }
  T unset(string* key, T value) {

    T res = value;
    for (int k = 0; k < this->size; k++) {
      if (this->keys[k] == key) {
        this->keys[k] = 0;
        return values[k];
      }
    }
    return res;
  }
  void apply(T (*func)(T, T), T v) {
    for (int k = 0; k < this->size; k++) {

      T value = (*func)(this->values[k], v);
      this->values[k] = value;
    }
  }
};
```

Question 4 **[6]**

You are given the following implementation of a measurement in degrees Fahrenheit.

```cpp
#include "Quant.h"

class Measure {
private:
  Quant* input;
public:
  Measure();
  ~Measure();
  void set_input(Quant*);
  float get_temp();
};
```

```cpp
#include "Measure.h"

Measure::Measure() {}
Measure::~Measure() {}
void Measure::set_input(Quant* quant) {
  this->input = quant;
}
float Measure::get_temp() {
  float quant = this->input->get_quant();
  return quant / 2;
}
```

The class `Measure` is to be used in a new context where its signature and measurement unit do not match. Instead of a function member `get_temp`, a function member `get_celsius` is needed that provides the temperature in degrees Celsius. It has been decided that the Adapter pattern is to be used to realise a class `Celsius` with the suitable signature.

As an illustration of the expected behaviour of class `Celsius`, the following two code fragments should both end up with an identical temperature in degrees Celsius `temp_celsius`.

Without adapter:

```cpp
float temp_celsius =
  (measure->get_temp() - 32.0) * 5.0 / 9.0;
```

With adapter:

```cpp
float temp_celsius =
  celsius->get_celsius()
```

Complete the class below to implement the `Celsius` adaptor.

```cpp
#include "Measure.h"

class Celsius {
private:

  Measure* ms;
public:
  Celsius(Measure*);
  ~Celsius();
  bool has_input();
  float get_celsius();
};
```

```cpp
#include "Celsius.h"

Celsius::Celsius(Measure* ms): ms(ms) {};
Celsius::~Celsius() {}
bool Celsius::has_input() {
    return true;
}
float Celsius::get_celsius() {

  return (ms->get_temp() - 32.0) * 5.0 / 9.0;
}
```

Question 5 **[9]**

The concurrent programs P1 and P2 below each run two threads `t1` and `t2` that might output different integer numbers depending on their interleaving. Note, that the mutexes ensure that it is sufficient to consider interleaving of the blocks `{x = 1; x = x + 1;}` and `{x = x - 1; x = x - 2;}` in program P1, and the statements `x = 1;`, `x = x + 1;` and `x = x - 1;`, `x = x - 2;` in program P2.

Which numbers might be output by **both** program P2 **and** program P1?

Tick the corresponding boxes below.

- ☐ -1
- ☐ 0
- ☒ 1
- ☒ 2
- ☐ 3

Program P1

```
int x;
mutex m;

static void init() {
    for (int i = 0; i < 2; i++)
    {
        m.lock();
        x = 1;


        x = x + 1;
        m.unlock();
    }
}

static void decr() {
        m.lock();
        x = x + 1;


        x = x - 2;
        m.unlock();
}

int main () {
    x = 0;

    thread t1 (init);
    thread t2 (decr);

    t1.join();
    t2.join();

    cout << "x: " << x << endl;
}
```

Program P2

```
int x;
mutex m;

static void init() {
    for (int i = 0; i < 2; i++)
    {
        m.lock();
        x = 1;
        m.unlock();
        m.lock();
        x = x + 1;
        m.unlock();
    }
}

static void decr() {
        m.lock();
        x = x + 1;
        m.unlock();
        m.lock();
        x = x - 2;
        m.unlock();
}

int main () {
    x = 0;

    thread t1 (init);
    thread t2 (decr);

    t1.join();
    t2.join();

    cout << "x: " << x << endl;
}
```

Question 6 **[10]**

The matrix component implementation below contains an error. As soon as function member `compose` is called a deadlock occurs. In this case, the execution of a program calling `compose` is blocked.

```
#include <string>
using namespace std;

#include <mutex>
using namespace std;

class Matrix {
private:
    int rows;
    int cols;
    int* values;
    mutex m;
public:
    Matrix(int rows, int cols):
      rows(rows), cols(cols), values(new int[rows*cols]) {}
    ~Matrix() {}
    int put_entry(int row, int col, int val) {
        scoped_lock lock {m};
        int pos = row * this->rows + col;
        int res = this->values[pos];
        return res;
    }
    void clear() {
        scoped_lock lock {m};
        for (int k = 0; k < this->rows*this->cols; k++) {
          this->values[k] = 0;
        }
    }
    bool compose(int row, int col, int val) {
        scoped_lock lock {m};
        int old = this->put_entry(row, col, 0);
        if (old == 0) {
            this->put_entry(row, col, val);
        } else {
            this->put_entry(row, col, old);
        }
        return old == 0;
    }
};
```

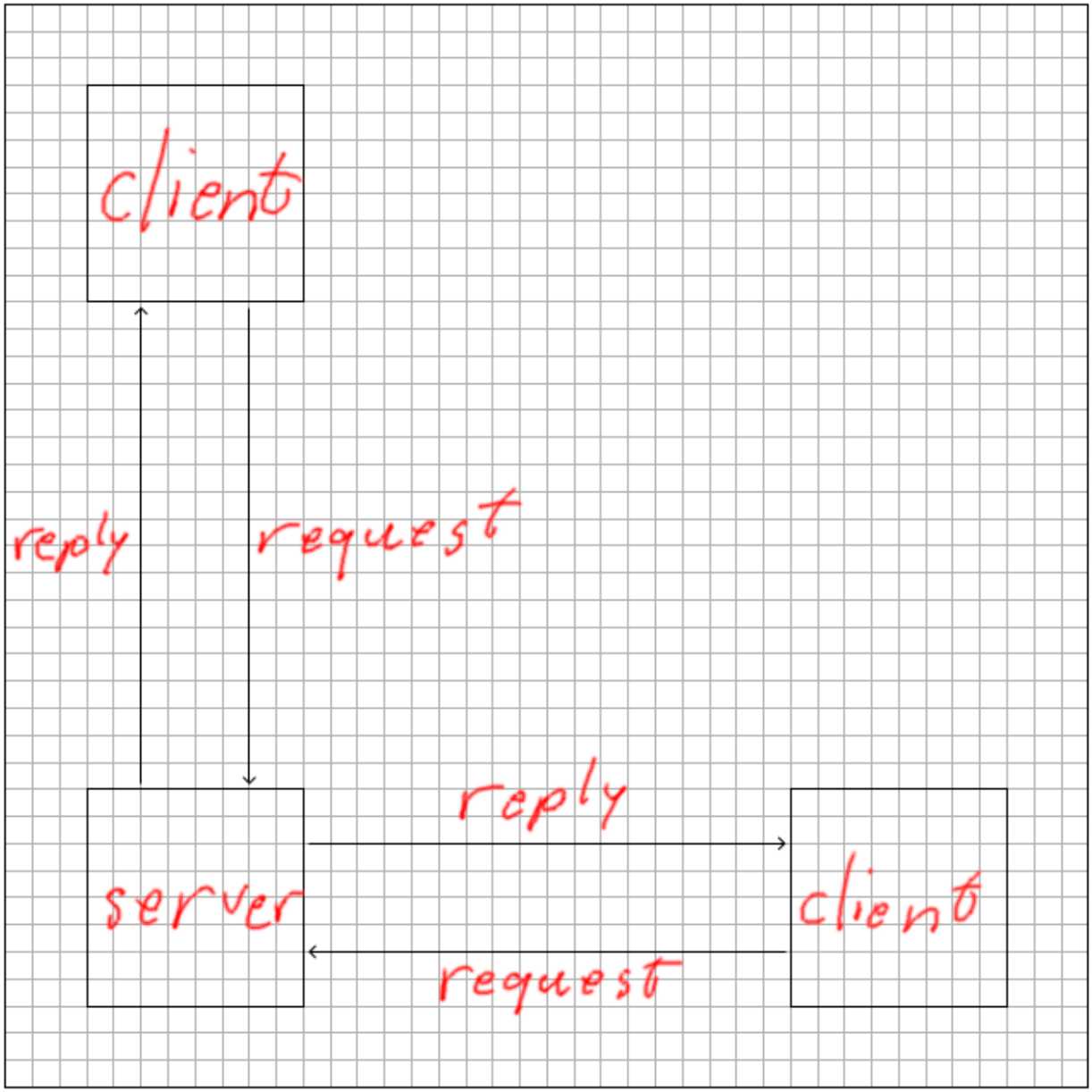
The statement of Question 6 is continued on the next page.

The problem can be fixed using the Thread-Safe Interface Design pattern. Apply the pattern to the class `Matrix` completing the class below at the indicated locations.

```
#include <string>
using namespace std;

#include <mutex>
using namespace std;

class Matrix {
private:
    int rows;
    int cols;
    int* values;
    mutex m;
    int _put_entry(____int row, int col, int val_____________________) {
        int pos = row * this->rows + col;
        int res = this->values[pos];
        return res;
    }
    void _clear() {
        for (int k = 0; k < this->rows*this->cols; k++) {
          this->values[k] = 0;
        }
    }
    bool _compose(_int row, int col, int val_____________________) {
        int old = this->_put_entry(row, col, 0);
        if (old == 0) {

          this->____put_entry(row, col, val)_____________________;
        } else {

          this->____put_entry(row, col, old)_____________________;
        }
        return old == 0;
    }
public:
    Matrix(int rows, int cols):
      rows(rows), cols(cols), values(new int[rows*cols]) {}
    ~Matrix() {}
    int put_entry(int row, int col, int val) {

        scoped_lock lock {m}_____________________________________;

        return this->_put_entry(row, col, val)___________________;
    }
    void clear() {

        scoped_lock lock {m}_____________________________________;

        this->_clear()___________________________________________;
    }
    bool compose(int row, int col, int val) {

        scoped_lock lock {m}_____________________________________;

        return this->_compose(row, col, val)_____________________;
    }
};
```

Question 7 **[4]**

In the documentation for a software architecture for a networked software system you see the following incomplete drawing. You know this is supposed to be a client-server architecture. Which ones are the clients and which one is the server and which messages are sent between them. Indicate this in the drawing by inserting the words “client” and “server” in the boxes, as well as “request” and “reply” at the edges.



Question 8 **[4]**

You are given the following program making use of a class `Stream`.

```
1  #include "Stream.h"
2
3  Files* files =
4    new Files(2, 2);
5  Stream* stream =
6    new Stream(files);
7
8  void open_files() {
9    files->open(0, "x.o", "rw");
10   files->open(1, "y.o", "rw");
11   files->wr[0] = false;
12   files->wr[1] = false;
13 }
14
15 int main () {
16   open_files();
17   stream->read(0);
18 }
```

```
19 #include "Files.h"
20
21 #define READ 1
22
23 class Stream {
24 private:
25   Files* files;
26 public:
27   Stream(Files*);
28   ~Stream();
29   int write(int, int, int);
30   int read(int);
31 };
```

```
32 #include "Stream.h"
33 #include <stdio.h>
34
35 Stream::Stream(Files* files): files(files) {}
36 Stream::~Stream() {}
37 int Stream::write(int i, int v, int r) {
38   this->files->wr[i] = true;
39   int x = fputc(v, this->files->get(i));
40   if (r == READ) {
41     x = fgetc(this->files->get(i));
42   }
43   return x;
44 }
45 int Stream::read(int i) {
46   if (!this->files->wr[i]) {
47     return fgetc(this->files->get(i));
48   }
49   this->files->wr[i] = false;
50   return 0;
51 }
```

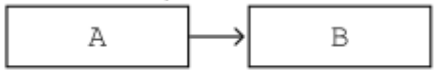
What kinds of coupling does the program contain and where?

- [x] Common Coupling — 3 to 6
- [x] Control Coupling — 40 to 42, 46 to 48
- [ ] Routine Call Coupling
- [x] Content Coupling — 11 to 12

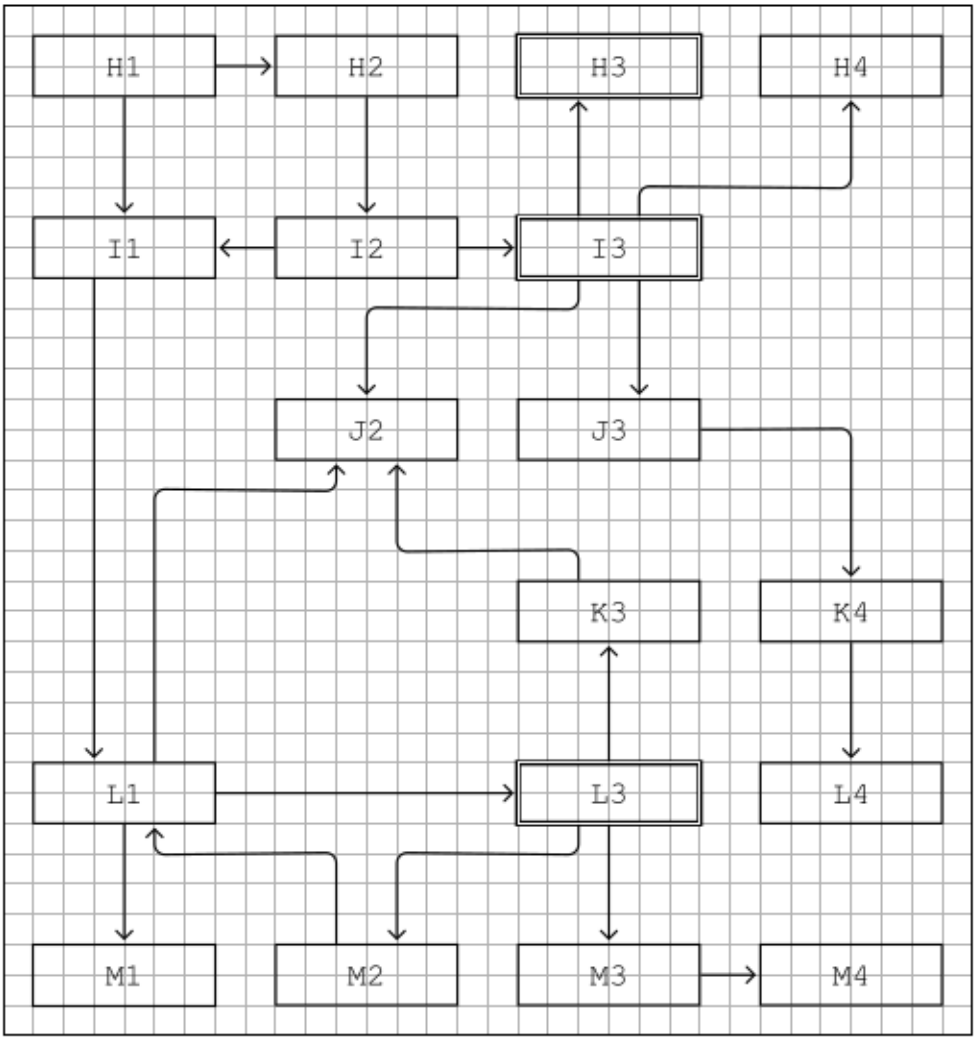
Tick the corresponding boxes on the left and indicate the line number ranges where the indicated kind of coupling can be observed on the right. In your answer refer to the line numbers of lines containing the corresponding statements.

Question 9 **[6]**

The figure below shows a diagram of the units implementing some application. The names of the units `H1`, `H2`, ... are shown in the rectangles representing the units. Edges in the diagram indicate which unit calls which. For instance,



describes that unit `A` calls unit `B`. Given the call dependencies in the diagram below, write down the names of units for which stubs and drivers need to be implemented for the unit tests of the units `H3`, `J3` and `L3`. Remember that a *stub* implements functionality of some unit *called* from the unit under test, and a *driver* implements functionality of some unit *calling* the unit under test.



Fill in the missing information on names of stubs and drivers in the table below. If there is more than one name, separate the names by commas. For a given unit, only the drivers and stubs connected directly to the unit by a single edge should be considered. Transitive relationships are to be disregarded.

Unit	Drivers	Stubs
H3	I3	
I3 ~~J3~~	I2	H3, H4, J2, J3
L3	L1	K3, M2, M3

Question 10 **[4]**

Function `mult` stated below implements polynomial multiplication.

```
float mult (float* p, float* q, int N) {
  float pl[N/2], ql[N/2], ph[N/2], qh[N/2], t1[N/2], t2[N/21];
  float r[2*N-2], rl[N], rm[N], rh[N];
  int i, N2;
  if (N == 1) {
    r[0] = p[0]*q[0];
    return r[0];
  }
  for (i=0; i<N/2 || N == 0; i++) {
    pl[i] = p[i]; ql[1] = q[i];
  }
  for (i = N/2; i < N; i++) {
    ph[i-N/2] = p[i]; qh[i-N/2] = q[i];
  }
  for (i = 0; i < N/2; i++) {
    t1[1] = pl[i]+ph[i];
    for (i = 0; i < N/2; i++) {
      t2[1] = ql[i]+qh[i];
      rm[i] = mult(t1, t2, N/2);
      rl[i] = mult(pl, ql, N/2);
      rh[i] = mult(ph, qh, N/2);
    }
  }
  for (i = 0; i < N-1; i++) {
    r[i] = rl[i]; r[N-1]= 0;
  }
  for (i = 0; i < N-1; i++) {
    r[N+i] = rh[i];
    for (i= 0; i<N-1; i++) {
      r[N/2+i] += rm[i] - (rl[i]+rh[i]);
    }
  }
  return r[0];
}
```

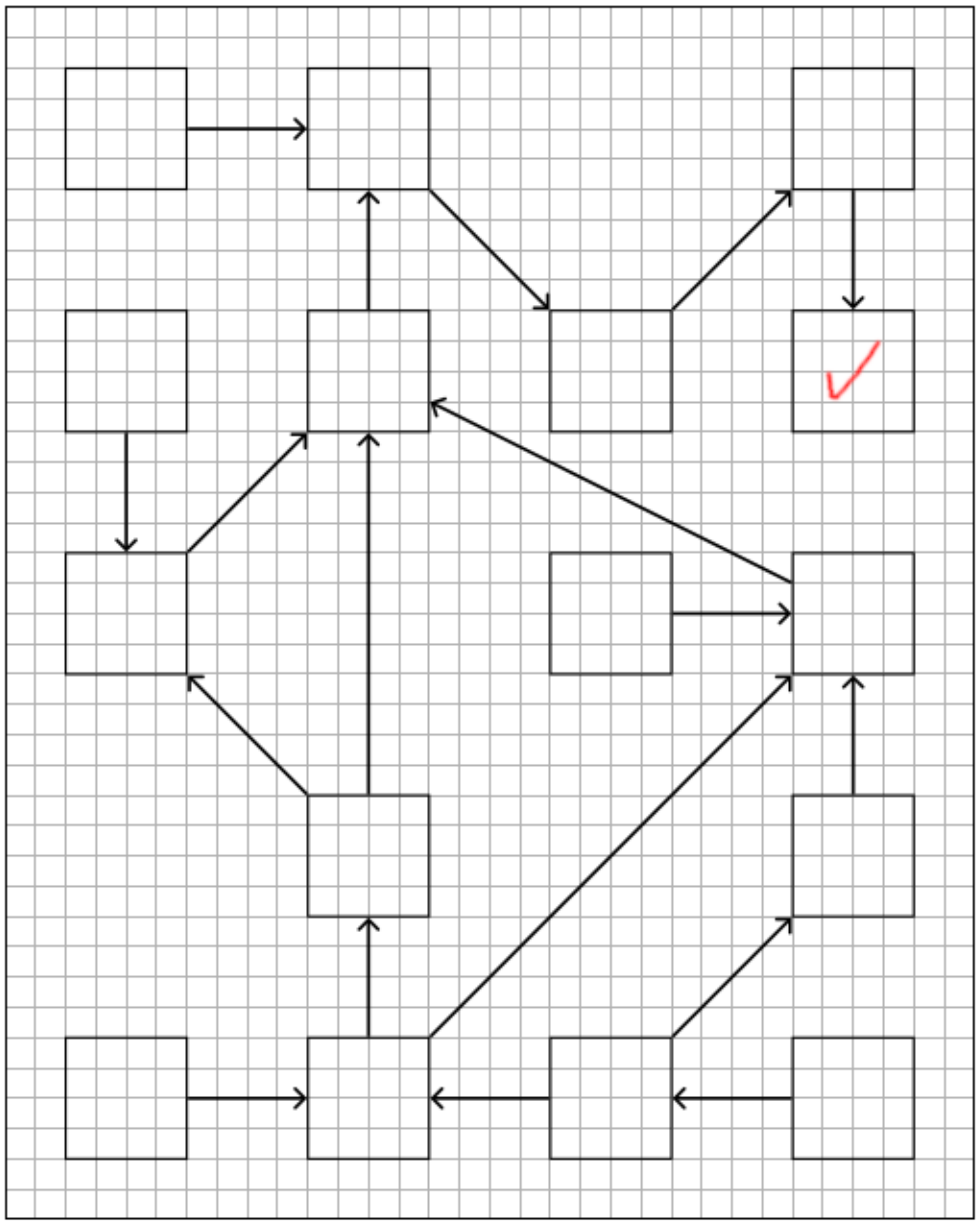
How many test cases do you need to write to cover all basis paths?

Answer: 10

Question 11 **[4]**

The figure below shows a diagram of the dependencies of the components of a software application. Dependencies between components are indicated by edges X → Y meaning that X depends on Y.

Assume that the diagram describes a Clean Architecture containing **exactly one** core component. Mark that core component in the diagram with a ✓ .



Question 12 **[6]**

Calculate the fan-in and the fan-out of the C-functions `MD2Update` and `MD2Transform` with respect to function calls. Assume that the C-functions `MD2Update` and `MD2Transform` are not called from any other function not shown below. Hint: You do not need to understand what these functions compute to calculate the fan-in and fan-out.

Function	Fan-in	Fan-out
`MD2Update`	0	2
`MD2Transform`	1	0

```c
void MD2Update(MD2_CTX *context;
               const unsigned char *input;
               unsigned int inputLen) {
{
  unsigned int idx, piece;
  for (idx = 0; idx < inputLen; idx += piece) {
    piece = 32 - context->i;
    if ((inputLen - idx) < piece)
      piece = inputLen - idx;
    memcpy(&context->X[context->i], &input[idx], piece);
    if ((context->i += piece) == 32)
      MD2Transform(context);
  }
}
```

```c
void MD2Transform(MD2_CTX *context) {
  u_int32_t l, j, k, t;

  for (l = context->C[15], j = 0; j < 16; j++) {
    context->X[32 + j] = context->X[j] ^ context->X[16 + j];
    l = context->C[j] ^= S[context->X[16 + j] ^ l];
  }

  for (t = j = 0; j < 18; t = (t + j) % 256, j++)
    for (k = 0; k < 48; k++)
      t = context->X[k] = (context->X[k] ^ S[t]);

  context->i = 16;
}
```

Question 13 **[5]**

You are given the following function `binsearch` that implements binary search.

```
1  /* Recursive binary search: */
2  int binsearch(int a[], int l, int r, int x) {
3      if (r >= l) {
4          int m = l + (r - l) / 2;
5          if (a[m] == x)
6              return m;      //<-- provide test case for this branch
7          if (a[m] > x) {
8              return binsearch(a, l, m - 1, x);
9          }
10         return binsearch(a, m + 1, r, x);
11     }
12     return -1;
13 }
```

For a call of the function above

```
int i = binsearch(a, l, r, x);
```

specify a test case that executes line 6 in the program above.

Insert you answer for the input values and the expected result below.

input values	a == {1, 2, 3}
	l == 0
	r == 2
	x == 2
expected result	i == 1

Question 14 **[9]**

You are given an implementation of a stack machine that permits to execute commands on a stack: function member `push` puts a value at the top, `drop` removes all values, `dup` duplicates the top element, `sum` takes the sum of all values and `get` returns the machine's result.

```
class Stack_machine {
private:
  // maximal size of the stack
  int size;
  // index of top most element
  int top;
  // buffer holding the stack
  int* stack;
public:
  Stack_machine(int s) {
    this->size = s;
    this->top = 0;
    this->stack = new int[s];
  }
  ~Stack_machine() {
    delete[] this->stack;
  }
  void push (int v) { // pushes v on
                      // the stack
    if (this->top < this->size) {
      this->top = this->top + 1;
      this->stack[this->top] = v;
    }
  }
  void drop() { // drops all values
                // from the stack
    top = -1;
  }
  void dup() { // duplicates the top
               // most value
    if (this->top >= 0 &&
        this->top < this->size) {
      this->stack[this->top + 1] =
        this->stack[this->top];
      this->top = this->top + 1;
    }
  }
  void sum() { // sums up the values
               // of the stack
    while (this->top > 0) {
      this->stack[this->top - 1] =
        this->stack[top] +
        this->stack[top - 1];
      this->top = this->top - 1;
    }
  }
  int get() { // returns the result
              // computed by
              // the machine
    if (this->top >= 0) {
      return this->stack[top];
    } else {
      return 0;
    }
  }
};
```

Specify the following three behavioural properties (in terms of the signature of the class) that we **would like** the `Stack_machine` class to satisfy.

1: Given a machine, dropping all values from the stack, the result computed by the machine is `0`.

2: Given a machine, pushing `1` on the stack, then pushing `2` and then summing up, the result is at least `3`.

3: Given a machine, pushing `1` on the stack, then duplicating the top most value and then summing up, the result is at least `2`.

Specification 1:	Specification 2:	Specification 3:
Stack_machine m; int s = 10;	Stack_machine m; int s = 10;	Stack_machine m; int s = 10;
m = new Stack_machine(s);	m = new Stack_machine(s);	m = new Stack_machine(s);
m.drop();	m.push(1);	m.push(1);
	m.push(2);	m.dup();
ASSERT(m.get() == 0)	ASSERT(m.sum() >= 3)	ASSERT(m.sum() >= 2)

Question 15 **[9]**

You are given the following class that maintains a histogram of integer indices. For each index it stores the count of how often that index has been pushed. Some additional member functions permit manipulating the histogram.

```
class Histogram {
private:
  int n;        // size of the buffer
  int* values; // array holding the buffer elements
  int imax;     // index of a current maximum count
public:
  void clear() {     // Clears the histogram
    for (int k = 0; k < this->n; k++) {
      this->values[k] = 0;
    }
  }
  Histogram(int s): n(s), values(new int[s]), imax(0) {
    this->clear();
  }
  ~Histogram() { delete[] values; }
  int max() {        // Returns the index of the current maximum count
    return imax;
  }
  int size() {
    return n;
  }
  int get(int i) {  // Returns the current count stored for index i
    if (0 <= i && i <= this->n)
      return values[i];
    return 0;
  }
  void put(int i) { // Increases the count for index i by 1
    if (0 <= i && i < this->n)
      this->values[i] = this->values[i] + 1;
    if (this->values[i] > this->values[imax])
      imax = i;
  }
  void shrink() {   // Shrinks all counts larger than 0 by 1
    for (int k = 0; k < this->n; k++) {
      if (this->values[k] > 0)
        this->values[k] = this->values[k] - 1;
    }
  }
  bool empty() {    // Returns whether any stored count is larger than 0
    for (int k = 0; k < this->n; k++)
      if (this->values[k] > 0)
        return false;
      return true;
  }
};
```

The statement of Question 15 is continued on the next page.

Specify the following invariant and two contracts by means of the function members of the class `Histogram` above.

Invariant:

(1) For `h` of type `Histogram`, the index of the current maximum count is larger or equal to `0` and smaller than the histogram's size.

For h of type Histogram

h.max() >= 0 && h.max() < h.size();

Contracts:

(2) For an empty `h` of type `Histogram`, an index `x` with `0 <= x`, `x < h.size()` and any index `y`, increasing the count of `x`, then increasing the count of `y`, then again increasing the count of `x`, the index with maximum count is `x`.

For h of type Histogram with h.empty() == true, an index x with 0 <= x && x < h.size(), and any index y

h.put(x); h.put(y); h.put(x); yields

h.max() == x;

(3) For an empty `h` of type `Histogram` and indices `x != y`, increasing the count of `x`, then increasing the count of `y`, then shrinking the counts larger than `0`, yields an empty histogram.

For h of type Histogram with h.empty == true, and indices x, y with x != y

h.put(x); h.put(y); h.shrink(); yields

h.empty == true;